

# Project 5 — ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker

---

**Title:** ContentForge Pro Production Ready Content Pipeline With API Database And Docker

**Type:** Multi-agent (Project 4's 5 agents, made production-ready) · **Stack:** Python, LangGraph, FastAPI, SQL, Docker, testing/MLOps · **Level:** Advanced / Capstone **Tagline:** ContentForge (Project 4), served as a real API — saved, containerized, tested, and defendable under a security/scaling review.

## Overview

---

ContentForge Pro takes Project 4's finished 5-agent system and turns it into a real, deployable service: served over a REST API, backed by a database, containerized, rate-limited, and gated by an automated test suite before any new version can go live. It solves a real problem: a multi-agent system that only runs when someone opens a terminal cannot be used by a team, called by another service, or trusted with real traffic. No new agents are built here — the five agents from Project 4 are already finished; this project is entirely about running them safely, which is the gap between a tutorial project and a hireable, production-grade one.

## Features

---

- REST API (FastAPI) exposing the ContentForge graph over HTTP with typed request/response models and safe error handling
- Every run persisted to a database and retrievable by ID, with a per-request ID threaded through every log line
- Dockerized deployment that runs identically on any machine, with secrets passed only via environment variables at run time — never baked into the image
- In-memory rate limiting using a rolling request-timestamp window, closing the gap a fixed-reset counter leaves open
- Automated pytest-based test suite (including an LLM-judge check) that gates any new version before it's allowed to go live
- Rollback function that restores the last version that actually passed the test suite, scanning back through failed versions if needed
- Structured, request-tagged logging good enough to debug a bad production answer after the fact

## Tech Stack

---

- Python
- LangGraph
- FastAPI
- SQL (SQLAlchemy or SQLite)
- Docker
- pytest
- OpenAI API

## Architecture

---

FastAPI exposes a `POST /run-task` route that calls Project 4's existing 5-agent LangGraph graph directly — none of the agents themselves change. Every run is written to the database as `"pending"` and then updated with its result, and can be looked up later via `GET /runs/{id}`. The whole app is packaged in a Docker container with secrets injected only at runtime, and protected by rolling-window rate limiting. A pytest-based suite, including an LLM-judge check, must pass before a new prompt or setting version is allowed to go live, and a rollback function restores the last version that actually passed — not just whatever came before it.

**Jump to:** [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

## Charter (what this project is)

---

Project 4, made production-ready: served over FastAPI, saved to a real database, containerized, tested with a real test suite, watchable, and designed from unclear business requirements instead of a spec handed to you already solved. This is the portfolio-level deliverable, and the final proof of the whole curriculum. **No new agents get built here** — all 5 of ContentForge's agents are already done; this project is entirely about running them safely.

## The Story — what you're actually building

---

Picture ContentForge (Project 4) as a very capable employee who, so far, only knows how to work if you sit at their desk and run their code by hand. They do good work, but nobody else at the company can use them: there's no phone number to call them on, no record of what they've done, and no way to trust them with real customers hitting them all at once. Project 5 is the process of turning that one capable employee into a real, dependable service the rest of the company — or the internet — can actually rely on.

Concretely: Step 1 gives ContentForge a phone number — an HTTP address anyone can call with a topic and get an article back, instead of needing to run a Python script themselves. Step 2 gives it a filing cabinet — every run gets saved to a real database, so "what happened on that run from yesterday" has an actual answer instead of a shrug. Step 3 puts it in a shipping container — quite literally, a Docker container — so it runs identically on your laptop, a teammate's laptop, or a real server, and adds a bouncer at the door (rate limiting) so one abusive caller can't take the whole thing down. Step 4 adds a quality inspector who checks every new version before it's allowed to go live, plus a way to instantly undo a bad change — so improving ContentForge later never risks quietly breaking what already works.

None of these four steps touch the five agents themselves — Research, Analysis, Writer, Reviewer, and the Supervisor are already finished, from Project 4. This project is entirely about the safety net *around* them: the difference between "a cool thing I built on my computer" and "a system I could actually hand to someone else and defend under hard questions." That difference is what makes this the capstone.

**What you're actually building, in one line:** Project 4's 5-agent graph, wrapped behind a real HTTP API, backed by a database, containerized, and covered by a test suite that gates new versions before they go live.

**Why this needs to exist:** a multi-agent system that only runs when someone opens a terminal and runs a Python script by hand cannot be used by a team, called by another service, or trusted with real traffic — real use needs a stable address, a saved history, and a way to catch a bad change before it ships.

**When you'd reach for this at a real job:** whenever a working prototype needs to become a real service other people or systems can call — this is what "shipping" an AI agent actually looks like, not building a new agent.

**How it works, mechanically:** FastAPI exposes ContentForge's graph over HTTP, every run's input and output gets saved to a database row, Docker packages the whole thing to run the same way everywhere, and a pytest-based suite (including an LLM-judge check) has to pass before a new version is allowed to replace the one already running.

**Why not just do it some simpler/different way:** why not just keep running it as a local script and skip the API, database, and Docker layer? Because then only you, on your own machine, can ever use it — nobody else can call it, nothing gets recorded, and a laptop-only setup doesn't behave the same way on a real server. Why not just skip the test suite and check new changes by eye? Because eyeballing output doesn't scale and doesn't catch a quiet regression — a change that makes answers subtly worse can pass a quick manual look and still be worse, which is exactly what an automated test gate is built to catch.

## Why This Project Is Good for Your Portfolio

---

- **What problem it solves:** turns Project 4 from "works when I run it on my computer" into a system someone else could actually deploy, call over HTTP, trust with real traffic, and operate — the single biggest gap between a tutorial project and a hireable portfolio piece.
- **Why it matters:** this is the project you link in your resume/portfolio, not just describe — a live (or easy-to-run) API, a Dockerfile that builds and runs cleanly, a test suite that would catch a real regression, and a defensible answer to "how would this scale" or "what would you cut under a cost limit." This is what separates "I followed a LangGraph tutorial" from "I can be trusted to own a production AI system."
- **When you'd build something like this at a real job:** this *is* the shape of a real production AI engineering job — wrapping an agent system in a service, saving its history, testing it, watching it, and being able to defend every design trade-off to a skeptical senior engineer or a security review.
- **How it's built:** Project 4's multi-agent graph, served behind FastAPI, backed by a database for run history, containerized with secrets handled correctly, tested with an LLM-judge test suite, and designed from unclear business requirements instead of a spec handed to you already solved.

Problems you'll likely run into, and how to fix them:

| Problem                                                    | Fix                                                                                                                                                     |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| A secret ends up baked into a Docker image layer           | Never COPY a .env file into the image; pass secrets in only through docker run -e / environment variables at run time, and check with docker history    |
| An internal error shows a raw error trace to an API client | Catch it broadly at the route level, log the full detail on the server, return a plain, safe message with the right 5xx code                            |
| The test suite still passes after a prompt gets worse      | The suite has a hole — if a change you made worse on purpose doesn't drop the score, fix the test rules or judge prompt before trusting the suite again |
| Database connections leak under repeated requests          | Use connection pooling / properly closed sessions; recreate the leak under real load before saying it's fixed                                           |

Setup (do this once, before Step 1)

```
cd project_5_contentforge_pro
python -m venv .venv
source .venv/bin/activate
pip install openai langchain langchain-openai langgraph fastapi "uvicorn[standard]"
pip freeze > requirements.txt
```

New here vs. Project 4: fastapi + uvicorn (the API layer), sqlalchemy (database — or use the built-in sqlite3 directly, your call, Doc12 covers the trade-off), pytest (Doc13's test suite).

You'll also need [Docker installed](#) on your computer for Step 3 — this is a one-time setup on your machine, not a pip package.

## Built During These Documents

---

[12\\_production\\_engineering](#) → [13\\_testing\\_evaluation\\_observability](#) → [14\\_debugging\\_lab](#) → [16\\_system\\_design\\_architecture](#) → [17\\_interview\\_preparation](#) → [18\\_capstone](#)

## Plan Before You Code

---

This project *is* the plan-before-you-code process, at full scale. See [18\\_capstone/README.md](#): you'll be given business/functional/non-functional requirements, limits, scale, security, and cost targets — not a design. Write your design first; I review it like a senior architect at each stage.

## How To Build This — Step by Step

---

Unlike Projects 1-4, these steps aren't about building more agents — ContentForge's 5 agents are already finished. Each step here makes the *system around* them safer, one production concern at a time, so a failure at any step can be traced to that one step.

### Step 1 — ContentForge, Reachable Over HTTP for the First Time

*Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 1 of 4: ContentForge, Reachable Over HTTP for the First Time*

**What this step does:** proves Project 4 can be reached over HTTP at all, with none of the other production concerns yet — no database, no Docker, no login. Keeps "does the API layer work" separate from everything else. **Why this step matters:** every later step in this project builds on the assumption that the API layer itself is solid — if a database bug or a Docker bug shows up later, this step is what lets you rule out "is it even the HTTP layer" immediately. **Read first:** [12\\_production\\_engineering Core Concepts — "FastAPI"](#).

What to do:

1. Write one FastAPI route, `POST /run-task`, that calls Project 4's graph directly and returns the result — kept only in memory, nothing saved yet. Wrap the graph call in a try/except that returns a plain, safe error message on failure — a raw Python traceback handed back to an API caller is a real information leak, not just an ugly error page.
2. Add typed request/response Pydantic models, and confirm FastAPI's automatic 422 error on a bad request.
3. Test it by actually running `uvicorn` locally and hitting the route with `curl` or the auto-built `/docs` page — not just reading the code.

**What's new vs. Project 4:** ContentForge is now reachable over HTTP, not just from a terminal script. **What stays the same:** the graph itself — Step 1 doesn't change any agent.

**Stuck on this step?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

**When you'll hit this for real:** the exact moment any project moves from "works on my machine" to "someone else needs to call this" — every real AI engineering job includes this step.

**Your files after Step 1:**

```
project_5_contentforge_pro_production/
├─ api/
│   ├── main.py           → FastAPI app
│   ├── routes.py         → POST /run-task, GET /health
│   └─ schemas.py         → request/response Pydantic models
└─ (project_4_contentforge's graph, imported, unchanged)
```

## Step 2 — Every Run Saved to a Database and Findable by ID

*Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 2 of 4: Every Run Saved to a Database and Findable by ID*

**What this step does:** makes runs durable and traceable — a run's result now survives a restart and can be looked up later, which is the real requirement for the testing/rollback work in Step 4. **Why this step matters:** Step 4's test suite and rollback both need a real record of what happened on past runs to compare against — without this step's database, there's nothing for either of them to check.

**Read first:** [12\\_production\\_engineering Core Concepts — "Database storage"](#).

What to do:

1. Add a SQLite `Run` table (input, output, log, status, timestamps) and write to it on every request. Write the row as `"pending"` before calling the graph, then update it after — that way a lookup on a run that's still in progress returns an honest "pending" instead of looking like it never happened.
2. Add a `GET /runs/{id}` route to read a run back.
3. Add an ID for each request, threaded through every log line, so one run's logs are easy to search for afterward.
4. Test it: kill the database connection in the middle of a request (see Doc12's Break-It) and confirm a clean 500, not a raw error trace.

**What's new vs. Step 1:** every run is now saved and can be looked up on its own; logs are tagged per request. **What stays the same:** the API from Step 1 — `POST /run-task`'s request/response shape doesn't change, it just now has a side effect (a database write).

**Stuck on this step?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

**When you'll hit this for real:** the first time someone asks "what happened on that run from yesterday" and your honest answer is "I don't know, it's gone" — this step is what prevents that conversation.

### Your files after Step 2:

```
project_5_contentforge_pro_production/
├── api/
│   ├── main.py
│   ├── routes.py      → now writes to the database, includes GET /runs/{id}
│   └── schemas.py
├── db/
│   ├── models.py      → the Run table
│   └── database.py
└── (ContentForge's graph, unchanged)
```

## Step 3 — Containerized and Rate-Limited: Runs the Same Everywhere, Survives Abuse

*Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 3 of 4: Containerized and Rate-Limited: Runs the Same Everywhere, Survives Abuse*

**What this step does:** makes the app portable and hard to abuse — anyone can run it the exact same way from a container, and one client can't take it down by hammering it with requests. **Why this step matters:** "works on my machine" is not a claim anyone else can verify or rely on — a container plus rate limiting is what turns this from a project only you can run into one anyone can run and trust under real traffic. **What's new vs. Step 2:** a working `Dockerfile` and basic rate limiting get added. **What stays the same:** the API and database schema from Steps 1-2 — this step wraps operational safety around them, it doesn't redesign them. **When you'll hit this for real:** handing your project to anyone else to run — a teammate, a hiring manager, a server that's never seen your code before. "It works on my machine" stops being an acceptable answer here. **Read first:** [12\\_production\\_engineering Core Concepts — "Docker"](#).

What to do:

1. Write a `Dockerfile` ; confirm the app runs the same from a container, with only environment variables passed in at `docker run` time — no secret baked into any layer (check with `docker history` ).
2. Add basic in-memory rate limiting on `POST /run-task` . Prefer tracking actual request timestamps in a rolling window over a counter that resets on a fixed timer — a fixed-reset counter lets a client send double the intended rate right at the boundary where the counter resets.
3. Test it: the secret-in-image check with `docker history` , a missing-env-var container startup failure, and hitting the rate limit with quick repeated requests.

**Stuck on this step?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

### Your files after Step 3:

```
project_5_contentforge_pro_production/
├── api/
│   ├── main.py
│   ├── routes.py          → now rate-limited
│   └── schemas.py
├── db/
│   ├── models.py
│   └── database.py
├── Dockerfile
└── .dockerignore
```

## Step 4 — Test-Gated and Ready to Roll Back: No Regression Ships Quietly (final)

*Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 4 of 4: Test-Gated and Ready to Roll Back: No Regression Ships Quietly*

**What this step does:** the real "safe to keep changing" layer — a test suite that would catch a bad prompt change, and a release gate that refuses to let it go live. **Why this step matters:** every project up to this point proved things worked *once* — this step is what lets you keep improving ContentForge afterward without each change being a gamble on whether you quietly broke something that used to work. **What's new vs. Step 3:** a test suite (Doc13) and a versioned release gate + rollback (Doc19) get added on top of the containerized app. **What stays the same:** everything from Steps 1-3 — this step doesn't touch the API, database, or Docker setup, it adds a safety net around future changes to them. **When you'll hit this for real:** the day after this project is "done," when you want to change a prompt without risking breaking what's already working — this is the difference between "shipped once" and "safe to keep improving." **Read first:** [13\\_testing\\_evaluation\\_observability Core Concepts](#), [19\\_mlops\\_llmops Core Concepts](#).

What to do:

1. Build the Doc13 test suite against ContentForge's outputs — a fixed set of tasks with clear pass/fail or scored rules.
2. Build the Doc19 release gate on top of it — a new prompt/setting version can't go "live" without passing the suite.
3. Build the Doc19 rollback function — goes back to the last version that actually passed, not just "whatever came before." Store every version's suite result, including failed ones, and have rollback scan backward through however many failed versions it takes — the version immediately before the current one isn't necessarily one that ever passed.
4. Test it: a version made worse on purpose gets refused by the release gate, and a rollback correctly restores the last version that passed.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Your files after Step 4 (final):

```
project_5_contentforge_pro_production/
├── api/
│   ├── main.py
│   ├── routes.py
│   └── schemas.py
├── db/
│   ├── models.py
│   └── database.py
├── eval_suite/           → from Doc13
├── versioning.py         → from Doc19
├── deploy_gate.py        → from Doc19
├── rollback.py           → from Doc19
├── Dockerfile
├── .dockerignore
└── test_api.py
```

**Final Deliverable: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker** — ContentForge's 5 agents, served over FastAPI, saved to a database, containerized with secrets handled correctly, rate-limited, and protected by a test-gated release process with rollback.

## Checklist Before You Call This Done (matches the linked docs for full detail)

- [ ] The multi-agent system (from Project 4) served over FastAPI with proper request/response models and error handling
- [ ] A working database for run/conversation history
- [ ] Containerized with a correct Dockerfile — no secrets baked into the image
- [ ] The human-approval step kept, from Project 3/4, where it applies
- [ ] A runnable test suite that would actually catch a real regression
- [ ] Structured logging + tracking, good enough to debug a bad production answer after the fact
- [ ] Basic rate limiting, retries, and timeouts on every outside call
- [ ] You can defend cost, scaling, and security trade-offs when pushed on them directly
- [ ] Meets every item in [CURRICULUM.md §6](#), "Final capabilities"

## Suggested files

See the file tree in [12\\_production\\_engineering/README.md](#) as your starting point; it grows through Docs 13-18.

## Status

---

Not started. Track your progress in [../PROGRESS.md](#).

---

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**

---

*Part of a 13-project multi-agent AI engineering curriculum. See the [full curriculum](#) for the complete learning path and all other projects.*